PRAXIS ENGINEERING PLATFORM • INDUSTRY READINESS SERIES

# Cloud Engineer & SRE Systems Handbook

## Cloud Infrastructure, Kubernetes, Zero-Downtime Deployments, Observability & Incident Triage

A definitive field reference for junior and transitioning Cloud & Platform Engineers mastering Kubernetes orchestrations, Infrastructure-as-Code (Terraform), production observability, incident triage, and reliability engineering.

| | |
|---|---|
| **Curriculum Track:** | Cloud Engineer & SRE Systems Handbook |
| **Audience:** | Junior Engineers, Transitioning Developers, Technical Interns |
| **Platform Version:** | Praxis Workplace Simulation v2.4 (Enterprise Edition) |
| **Core Assessment:** | 5 MEQ Knowledge Check & 40 Production Engineering Problems |
| **Publication Date:** | October 2026 Edition • Confidential & Proprietary |

**NOTICE TO LEARNERS:** This handbook serves as the foundational text for the Praxis Workplace Simulation. Thorough review of all 10 chapters is required prior to taking the 5-Question Readiness Assessment and accessing the 40 Production Problems (20 Basic with Help Statements + 20 Intermediate).

# TABLE OF CONTENTS & CURRICULUM SYLLABUS

## Overview of the 10 Core Knowledge Domains in this Handbook


| Ch. | Chapter Title | Target Page |
|---|---|---|
| 1 | **Cloud Engineering Hierarchy, Platform Teams & Shared Responsibility**<br>Operating high-availability infrastructure across public clouds and hybrid environments. | Page 3 of 12 |
| 2 | **SRE Cadence, Toil Reduction & Error Budget Policies**<br>Balancing velocity against stability through mathematical error budgets. | Page 4 of 12 |
| 3 | **Infrastructure-as-Code (IaC), Terraform & GitOps Workflows**<br>Declaring cloud infrastructure as auditable, version-controlled state. | Page 5 of 12 |
| 4 | **Kubernetes Architecture, Pod Lifecycles & Networking**<br>Mastering container orchestration, resource limits, and service meshes. | Page 6 of 12 |
| 5 | **CI/CD Deployment Strategies: Rolling, Canary & Blue/Green**<br>Eliminating deployment downtime through progressive traffic shifting. | Page 7 of 12 |
| 6 | **The Observability Triad: Prometheus, Grafana & Distributed Tracing**<br>Transforming system telemetry into actionable operational intelligence. | Page 8 of 12 |
| 7 | **Incident Command System (ICS), Severity Levels & War Rooms**<br>How high-performing engineering teams organize during live production crises. | Page 9 of 12 |
| 8 | **Cloud Security, Zero Trust, IAM & Secrets Management**<br>Enforcing Principle of Least Privilege across cloud environments. | Page 10 of 12 |
| 9 | **Cloud & SRE Lexicon: 40 Essential Infrastructure Terms**<br>The essential vocabulary required to manage distributed cloud systems. | Page 11 of 12 |
| 10 | **The First 90 Days Survival Guide for Junior Cloud & SRE Engineers**<br>Your roadmap for navigating production access, avoiding outages, and earning trust. | Page 12 of 12 |


***Learning Invariant:*** *The 5-question knowledge check draws questions directly from these 10 chapters. Each syllabus topic corresponds to practical scenarios in the 40 production problems.*

# CHAPTER 1: CLOUD ENGINEERING HIERARCHY, PLATFORM TEAMS & SHARED RESPONSIBILITY

## Operating high-availability infrastructure across public clouds and hybrid environments.

Modern Cloud and Site Reliability Engineers (SRE) operate as platform enablers. Under the Cloud Shared Responsibility Model, the cloud provider manages physical hardware, hypervisors, and data centers, while the cloud engineer manages network topologies (VPCs, CIDRs), IAM policies, container runtimes, encryption in-transit and at-rest, and operational reliability.

### FIELD STANDARD: THE SRE GOLDEN PRINCIPLE

*Treat operations as a software problem. If an administrative task must be executed more than twice, automate it with code or Terraform rather than performing manual console clicks.*

| Role | Primary Responsibility | Critical SRE Deliverable |
|---|---|---|
| Platform / SRE Director | Multi-region cloud budget, compliance & organizational SLOs | Reliability roadmaps, vendor contracts, disaster recovery drills |
| Cloud Architect | Global network topology, security boundaries, multi-cloud strategy | Reference architectures, VPC peering/transit gateway designs |
| Site Reliability Engineer (SRE) | Service Level Objectives (SLOs), automated remediation, error budgets | Alert tuning, blameless postmortems, capacity planning |
| DevOps / CI/CD Engineer | Deployment pipelines, container image caching, build runners | Fast automated PR gates, artifact signing, release automation |
| Security Engineer (SecOps) | Vulnerability scanning, least privilege IAM, zero trust policies | Kube-bench audits, secrets rotation, network security policies |

# CHAPTER 2: SRE CADENCE, TOIL REDUCTION & ERROR BUDGET POLICIES

## Balancing velocity against stability through mathematical error budgets.

Google SRE doctrine mandates that SREs spend no more than 50% of their engineering capacity on repetitive operational 'toil' (manual deployments, ticket handling, cert renewals). The remaining 50% must be dedicated to engineering projects (automation, reliability tooling, fault injection). When a service exhausts its monthly Error Budget, feature releases halt to prioritize hardening.

### FIELD STANDARD: THE ERROR BUDGET EQUATION

*If an API has a 99.9% availability SLO over 30 days, its allowed downtime is exactly 43.2 minutes. Every outage subtracts minutes from this budget.*

| SRE Ceremony | Frequency & Focus | Cloud Engineer's Deliverable |
|---|---|---|
| Error Budget Review | Monthly (60 mins) | Review SLO compliance per tier-1 service; decide feature freeze/unfreeze |
| On-Call Handover | Weekly (30 mins) | Review alerts fired, runbook gaps identified, ongoing degraded states |
| Production Readiness Review | Pre-launch gate | Verify health checks, Prometheus metrics, backup restores, runbook docs |
| Disaster Recovery GameDay | Quarterly (Half-day) | Simulate regional cloud failover or database master crash in staging |
| Blameless Postmortem | Post SEV-1 incident | Document root cause, timeline, contributing factors, and preventative PRs |

# CHAPTER 3: INFRASTRUCTURE-AS-CODE (IAC), TERRAFORM & GITOPS WORKFLOWS

## Declaring cloud infrastructure as auditable, version-controlled state.

Manual modifications made in cloud web consoles ('ClickOps') are catastrophic anti-patterns causing configuration drift. All infrastructure must be declared as code using Terraform, OpenTofu, or Pulumi. State files must be locked remotely with DynamoDB/S3 to prevent concurrent execution races. GitOps engines (ArgoCD / Flux) continuously reconcile Git state with live Kubernetes clusters.

**FIELD STANDARD: TERRAFORM STATE LOCKING INVARIANT**

*Never run 'terraform apply' locally against production. Apply changes through an auditable CI/CD runner with plan artifacts generated and peer-reviewed in pull requests.*

| IaC Concept | Purpose & Mechanism | Operational Best Practice |
|---|---|---|
| Remote State Backend | Stores Terraform state in S3 with KMS encryption | Enforce state versioning and DynamoDB lock tables to stop split-brain |
| Terraform Modules | Reusable infrastructure blueprints (VPC, EKS, RDS) | Version tag modules semantically; avoid hardcoding resource names |
| Drift Detection | Automated daily scheduled Terraform plans | Alert immediately when manual console changes diverge from Git repo |
| GitOps Reconciliation | ArgoCD watching Git repository branch | Automatically sync manifests; enforce self-heal against manual kubectl edits |

# CHAPTER 4: KUBERNETES ARCHITECTURE, POD LIFECYCLES & NETWORKING

## Mastering container orchestration, resource limits, and service meshes.

Kubernetes reconciles desired state against observed state via control plane controllers. Pods must define precise CPU/memory `requests` (used for scheduling) and `limits` (enforced by Linux cgroups). Exceeding memory limits triggers Linux Kernel OOM killer (Exit Code 137). Networking is governed by Ingress Controllers (NGINX/Traefik), ClusterIP services, and CoreDNS lookups.

**FIELD STANDARD: KUBERNETES RESOURCE LIMITS RULE**

*Always set memory requests equal to memory limits to prevent node overcommit evictions. Configure graceful termination with preStop hooks and terminationGracePeriodSeconds.*

| K8s Object | Operational Role | Common Failure Mode |
|---|---|---|
| Pod | Atomic unit of deployment containing 1+ containers | CrashLoopBackOff due to unhandled startup exceptions or missing secrets |
| Deployment | Manages replica sets, rolling updates, and rollbacks | Rollout stalls if maxSurge/maxUnavailable are misconfigured |
| Horizontal Pod Autoscaler | Scales replicas based on CPU, memory, or custom metrics | Flapping/thrashing: pods scaling up and down rapidly without cooldown |
| Service & Ingress | L4/L7 load balancing and external routing to pods | 502 Bad Gateway if pod readiness probes fail or port numbers mismatch |
| PersistentVolumeClaim | Binds external storage (EBS, Ceph) to stateful pods | VolumeAttachment lock errors during cross-AZ pod rescheduling |

# CHAPTER 5: CI/CD DEPLOYMENT STRATEGIES: ROLLING, CANARY & BLUE/GREEN

## Eliminating deployment downtime through progressive traffic shifting.

Zero-downtime releases require deliberate deployment strategies. Rolling updates replace old pods incrementally. Blue/Green deployments provision two identical environments and switch DNS/Load Balancer targets instantly. Canary releases route 1% to 10% of real production traffic to the new version while Prometheus continuously monitors error rates; automated rollbacks trigger if anomalies appear.

**FIELD STANDARD: CANARY ROLLBACK THRESHOLD**

*If Canary 5xx error rate exceeds baseline by 0.5% or p99 latency spikes > 200ms within 5 minutes, immediately abort traffic shift and reroute 100% to stable.*

| Strategy | Pros | Cons | Ideal Use Case |
|---|---|---|---|
| Rolling Update | Zero extra compute required; default in K8s | Multiple versions run simultaneously; requires backward-compatible DB | Standard stateless microservices |
| Blue / Green | Instant cutover and immediate one-click rollback | Doubles compute cost during deployment window | Monolithic apps and complex database migrations |
| Canary Release | Limits blast radius to small % of users; automated | Requires advanced ingress routing (Istio/Flagger) & telemetry | High-traffic Tier-1 APIs with strict SLA |
| Feature Flags | Decouples code deployment from feature release | Accumulates technical debt if flags are not cleaned up promptly | A/B testing, gradual product rollouts |

# CHAPTER 6: THE OBSERVABILITY TRIAD: PROMETHEUS, GRAFANA & DISTRIBUTED TRACING

## Transforming system telemetry into actionable operational intelligence.

Observability consists of three foundational pillars: Metrics (numerical time-series for alerting), Logs (contextual event records for forensics), and Distributed Tracing (request journey across microservices). The Four Golden Signals (Latency, Traffic, Errors, Saturation) provide immediate visibility into system health. Alerts must be actionable and route to on-call engineers via PagerDuty.

### FIELD STANDARD: ALERTING HYGIENE INVARIANT

*Never page an engineer for an informational warning. Page only if customer impact is occurring or imminent and human action is required immediately. Everything else belongs in Slack.*

| Golden Signal | Prometheus Metric Example | Alert Threshold Best Practice |
|---|---|---|
| Latency | histogram_quantile(0.99, rate(http_req_duration_seconds_bucket[5m])) | Alert when p99 latency > 1,000ms for 3 consecutive evaluation cycles |
| Traffic | sum(rate(http_requests_total[1m])) | Alert on anomalous traffic drop (> 50% drop vs 7-day seasonal baseline) |
| Errors | sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(...) * 100 | Alert when 5xx error rate > 1.0% over rolling 5-minute window |
| Saturation | node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes | Alert when available memory < 15% or disk usage > 85% |

# CHAPTER 7: INCIDENT COMMAND SYSTEM (ICS), SEVERITY LEVELS & WAR ROOMS

## How high-performing engineering teams organize during live production crises.

During a Severity-1 outage, normal organizational hierarchy dissolves. The Incident Commander (IC) takes absolute charge of the response. The IC assigns roles: Operations Lead (executes runbooks and changes), Communications Lead (updates status pages and executives), and Scribe (documents timeline). No change is executed without IC verbal approval.

### FIELD STANDARD: RULE OF INCIDENT TRIAGE

*Mitigate first, diagnose root cause offline. Restoring customer service (via rollback, pod restart, or traffic shed) takes precedence over finding the exact bug.*

| Severity | Definition & Impact | Response SLA | War Room Protocols |
|---|---|---|---|
| SEV-1 (Critical) | Core service down for all users; data loss or security breach | 5 min acknowledge, 15 min response | Live war room opened; IC appointed; hourly customer updates |
| SEV-2 (Major) | Significant degradation for major customer segment; workaround exists | 15 min acknowledge, 30 min response | Dedicated Slack incident channel; on-call team assembled |
| SEV-3 (Minor) | Low-impact edge-case bug; non-critical admin tool affected | Next business day response | Handled via standard Jira ticketing sprint backlog |

# CHAPTER 8: CLOUD SECURITY, ZERO TRUST, IAM & SECRETS MANAGEMENT

## Enforcing Principle of Least Privilege across cloud environments.

Cloud security operates under the Zero Trust principle: 'Never trust, always verify.' Cloud resources and human operators must be granted only the minimum permissions required (Principle of Least Privilege). Long-lived AWS access keys are strictly forbidden; short-lived tokens (AWS IAM Roles Anywhere / OIDC) must be utilized. HashiCorp Vault or AWS Secrets Manager injects dynamic credentials.

**FIELD STANDARD: ZERO PUBLIC S3 / DATABASE INVARIANT**

*No database, Redis cache, or storage bucket should ever be exposed to the public internet (0.0.0.0/0). Enforce VPC Private Subnets, Security Groups, and CloudTrail audit logging.*

| Security Dimension | Threat Vector | Mandatory SRE Control |
|---|---|---|
| IAM Permissions | Over-privileged admin credentials leaked in CI logs | Least-privilege scoped policies; enforce MFA and role assumption |
| Secrets Management | Plaintext API tokens committed to GitHub repository | HashiCorp Vault / AWS Secrets Manager; rotate secrets every 90 days |
| Network Perimeter | Unrestricted inbound port 22 (SSH) open to internet | Bastion hosts with AWS SSM Session Manager; zero direct public SSH |
| Container Vulnerabilities | Base images containing unpatched CVEs | Trivy / Clair automated scanning in CI; block image pushes on High/Critical |

# CHAPTER 9: CLOUD & SRE LEXICON: 40 ESSENTIAL INFRASTRUCTURE TERMS

## The essential vocabulary required to manage distributed cloud systems.

Mastering infrastructure terminology enables precise operational dialogue during incident triage, architecture design reviews, and vendor negotiations. Learn these 40 terms thoroughly.

**FIELD STANDARD: VOCABULARY STANDARD**

*Distinguish strictly between SLI (Service Level Indicator, what is measured), SLO (Service Level Objective, target goal), and SLA (Service Level Agreement, legal contract with penalties).*

| Term | Industry Definition & Practical Engineering Context |
|---|---|
| SLI / SLO / SLA | Indicator (metric), Objective (internal reliability target), Agreement (contractual commitment to customers with financial penalties). |
| Error Budget | The maximum allowable downtime or error rate a service can accumulate before feature deployments are frozen. |
| Toil | Repetitive, manual, automatable operational work devoid of enduring engineering value that scales linearly with service size. |
| Infrastructure as Code (IaC) | Managing and provisioning computer data centers through machine-readable definition files rather than manual configuration. |
| CrashLoopBackOff | A Kubernetes pod state where a container repeatedly crashes immediately after startup, prompting exponential backoff restarts. |
| OOMKilled (Exit 137) | The Linux kernel terminating a container process because its physical memory usage exceeded its cgroup memory limit. |
| Ingress Controller | A specialized load balancer that manages external HTTP/HTTPS traffic routing to services within a Kubernetes cluster. |
| Circuit Breaker | A resiliency pattern that prevents an application from repeatedly trying to execute an operation that's likely to fail. |
| Zero Trust | A security framework requiring all users and microservices to be authenticated, authorized, and continuously validated. |
| GitOps | A operational framework that takes DevOps best practices used for application development and applies them to infrastructure automation. |

# CHAPTER 10: THE FIRST 90 DAYS SURVIVAL GUIDE FOR JUNIOR CLOUD & SRE ENGINEERS

## Your roadmap for navigating production access, avoiding outages, and earning trust.

As a junior cloud engineer, your primary goal is to develop deep systems empathy, respect production blast radiuses, and master telemetry before executing state changes. Never run a command you do not understand on a production terminal.

### FIELD STANDARD: THE PRODUCTION TERMINAL SAFEGUARD

*Before pressing 'Enter' on any kubectl or terraform command in production, pause and ask: 'What happens if this fails? What is my rollback command?' Always have a peer verify your plan.*

| Timeline Milestone | Key Objective & Deliverable | Success Metric |
|---|---|---|
| Week 1: Tooling & Read-Only Access | Configure AWS CLI, kubectl contexts, VPN access, and audit Dashboards | Successfully queried Grafana metrics and Kubernetes pods in staging |
| Week 2: First Staging Deploy | Deploy a microservice update through CI/CD to staging; inspect logs | Executed clean deployment; verified health check endpoints |
| Day 30: Runbook Authoring | Identify a tribal knowledge task; author an official documented runbook | Runbook approved by senior SRE; used by team in production |
| Day 60: Shadow On-Call | Shadow senior SRE during primary on-call rotation; participate in triage | Assisted in writing 2 postmortems; accurately diagnosed alert causes |
| Day 90: Production Contributor | Safely execute Terraform PR adding new cloud infrastructure in prod | Zero downtime; plan reviewed and verified against drift |